The normal case, where you're already in the application's directory, would be as follows:

```
$ ruby -I lib test/test_roman.rb
```

This would be a good time to investigate using Rake to automate your testing, see Using the Rake Build Tool, on page 245.

Test Suites

After a while, you'll grow a decent collection of test cases for your application. You may well find that these tend to cluster: one group of cases tests a particular set of functions, and another group tests a different set of functions. If so, you can group those test cases together into *test suites*, letting you run them all as a group.

This is easy to do—just create a Ruby file that requires minitest/autorun and then requires each of the files holding the test cases you want to group. This way, you build yourself a hierarchy of test material.

- You can run individual tests by name.
- You can run all the tests in a file by running that file.
- You can group a number of files into a test suite and run them as a unit.
- You can group test suites into other test suites.

This gives you the ability to run your unit tests at a level of granularity that you control, testing just one method or testing the entire application.

Most people seem to use test_ as the test-case filename prefix, a sample test suite file might look like this.

```
require 'minitest/autorun'
require_relative 'test_connect'
require_relative 'test_query'
require_relative 'test_update'
require_relative 'test_delete'
```

Now, if you run Ruby on the this file, you execute the test cases in the four files you've required.

Testing with RSpec

The minitest framework has a lot going for it. It is simple and it is compatible in style with frameworks from other languages (such as JUnit for Java and pytest for Python).

RSpec has different things going for it. It's feature-rich (which some would say is another way of saying "complicated"), and it has a different vocabulary for discussing testing. It also has a different syntax; although, it's one that has influenced the design of other testing tools including the Jasmine and Jest JavaScript testing frameworks.

In RSpec, the focus is not on assertions. Instead, you write *expectations*. RSpec is very much concerned with driving the design side of things. As a result, the vocabulary of RSpec—expectation, specification—are all associated with ways you might reason about your code before you write it. A "spec" is something you'd write before coding, an "assertion" is something you use to describe code that already exists.